

430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Chapter 4: Parameters and Overloading
 - Call-by-value & Call-by-reference
 - Overloading

Outline

- Chapter 4: Parameters and Overloading
 - Call-by-value & Call-by-reference
 - Overloading

Ch 4. Parameters and Overloading

- Two methods of passing arguments as parameters to a function
- **Call-by-value**
 - "copy" of value is passed
- **Call-by-reference**
 - "address of" actual argument is passed

Call-by-Value Parameters

- A **copy** of actual argument is passed
 - Considered as a "local variable" inside the function
- If modified, only the **"local copy"** is changed
 - Function has no access to "actual argument" from the caller
- This is the default method
 - Used in all examples so far

Call-by-Value Example:

Display 4.1 Formal Parameter Used as a Local Variable (1 of 3)

Display 4.1 Formal Parameter Used as a Local Variable (part 1 of 2)

```
1  //Law office billing program.
2  #include <iostream>
3  using namespace std;

4  const double RATE = 150.00; //Dollars per quarter hour.

5  double fee(int hoursWorked, int minutesWorked);
6  //Returns the charges for hoursWorked hours and
7  //minutesWorked minutes of legal services.

8  int main( )
9  {
10     int hours, minutes;
11     double bill;
```

Call-by-Value Example:

Display 4.1 Formal Parameter Used as a Local Variable (2 of 3)

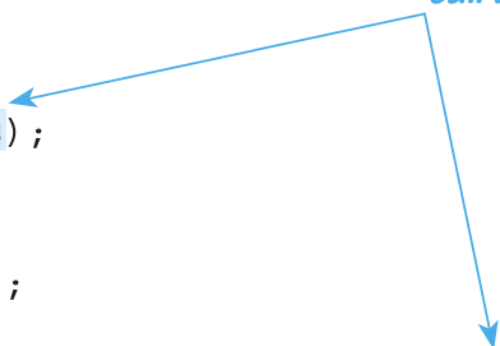
```
12     cout << "Welcome to the law office of\n"
13         << "Dewey, Cheatham, and Howe.\n"
14         << "The law office with a heart.\n"
15         << "Enter the hours and minutes"
16         << " of your consultation:\n";
17     cin >> hours >> minutes;

18     bill = fee(hours, minutes);

19     cout.setf(ios::fixed);
20     cout.setf(ios::showpoint);
21     cout.precision(2);
22     cout << "For " << hours << " hours and " << minutes
23         << " minutes, your bill is $" << bill << endl;

24     return 0;
25 }
```

The value of minutes is not changed by the call to fee.



Call-by-Value Example:

Display 4.1 Formal Parameter Used as a Local Variable (3 of 3)

```
26 double fee(int hoursWorked, int minutesWorked)
27 {
28     int quarterHours;
29     minutesWorked = hoursWorked*60 + minutesWorked;
30     quarterHours = minutesWorked/15;
31     return (quarterHours*RATE);
32 }
```

minutesWorked is a local variable initialized to the value of minutes.

(continued)

Sample Dialogue

```
Welcome to the law office of
Dewey, Cheatham, and Howe.
The law office with a heart.
Enter the hours and minutes of your consultation:
5 46
For 5 hours and 46 minutes, your bill is $3450.00
```


Call-by-Value Pitfall

- Common Mistake:
 - Declaring parameter "again" inside function:

```
double fee(int hoursWorked, int minutesWorked)
{
    int quarterHours;    // local variable
    int minutesWorked    // NO!
}
```

- Compiler error results
 - "Redefinition error..."
- Value arguments are like "local variables"
 - Function gets them "automatically"

Call-By-Reference Parameters

- Used to provide access to caller's actual argument
 - Caller's data can be modified by called function!
- Argument MUST be a variable, not a constant
- Typically used for input function
 - To retrieve data for caller
 - Data is then "given" to caller
- Specified by ampersand, **&**, after type in formal parameter list

Call-By-Reference Example:

Display 4.2 Call-by-Reference Parameters (1 of 3)

Display 4.2 Call-by-Reference Parameters

```
1  //Program to demonstrate call-by-reference parameters.
2  #include <iostream>
3  using namespace std;

4  void getNumbers(int& input1, int& input2);
5  //Reads two integers from the keyboard.

6  void swapValues(int& variable1, int& variable2);
7  //Interchanges the values of variable1 and variable2.

8  void showResults(int output1, int output2);
9  //Shows the values of output1 and output2, in that order.

10 int main( )
11 {
12     int firstNum, secondNum;

13     getNumbers(firstNum, secondNum);
14     swapValues(firstNum, secondNum);
15     showResults(firstNum, secondNum);
16     return 0;
17 }
```

Call-By-Reference Example:

Display 4.2 Call-by-Reference Parameters (2 of 3)

```
18 void getNumbers(int& input1, int& input2)
19 {
20     cout << "Enter two integers: ";
21     cin >> input1
22     >> input2;
23 }

24 void swapValues(int& variable1, int& variable2)
25 {
26     int temp;

27     temp = variable1;
28     variable1 = variable2;
29     variable2 = temp;
30 }
31
32 void showResults(int output1, int output2)
33 {
34     cout << "In reverse order the numbers are: "
35     << output1 << " " << output2 << endl;
36 }
```

Call-By-Reference Example:

Display 4.2 Call-by-Reference Parameters (3 of 3)

Sample Dialogue

```
Enter two integers: 5 6
```

```
In reverse order the numbers are: 6 5
```

Quiz1:

What would be the output if the `swapValues` is defined with call-by-value parameters? Namely, if we have

```
void swapValues(int variable1, int variable2) ?
```

Call-By-Reference Example:

Display 4.2 Call-by-Reference Parameters (3 of 3)

Sample Dialogue

Enter two integers: 5 6

In reverse order the numbers are: 6 5

Quiz2:

What would be the output if the first parameter in swapValues is called-by-value?
Namely, if we have

```
void swapValues(int variable1, int& variable2)?
```

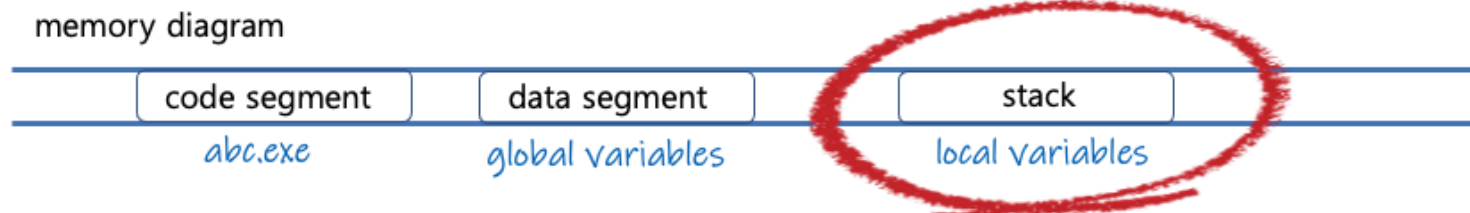
Call-By-Reference Details

- What's really passed in?
- A "reference" to caller's actual argument!
 - Refers to **memory location of the actual argument**
 - Called "address", which is a unique number referring to distinct place in memory
- Pros/Cons of call-by-value in contrast to call-by-reference
 - Pros: Variables in the calling func are not unexpectedly modified by the called func
 - Cons: Copying the argument to the parameter takes time

More on Call-by-value /reference

:What happens when a function is called?

```
void swap_using_value(int a, int b) { int tmp = a; a = b; b = tmp; }
void swap_using_ref(int& a, int& b) { int tmp = a; a = b; b = tmp; }
void f_with_const_ref(const int& a) { /* computation using a...*/ }
void main() {
    int u,v;
    u = 1, v = 2; swap_using_value(u,v);
    u = 1, v = 2; swap_using_ref(u,v);
    u = 1, v = 2; f_with_const_ref(u); std::cout << u << "\n";
}
```



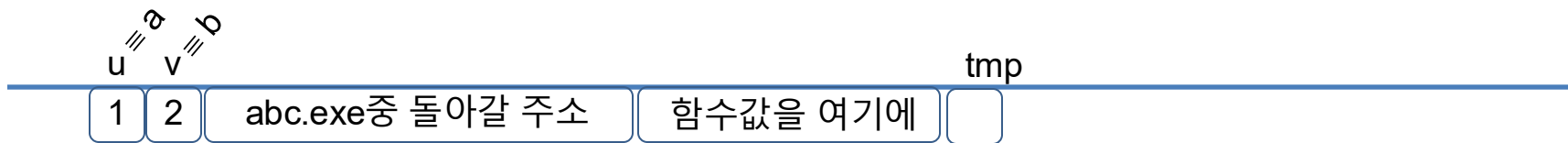
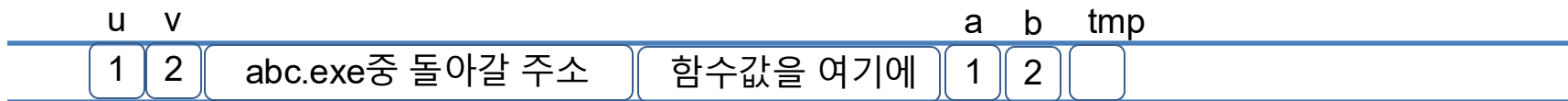
For the time being, we will show only the stack

More on Call-by-value /reference

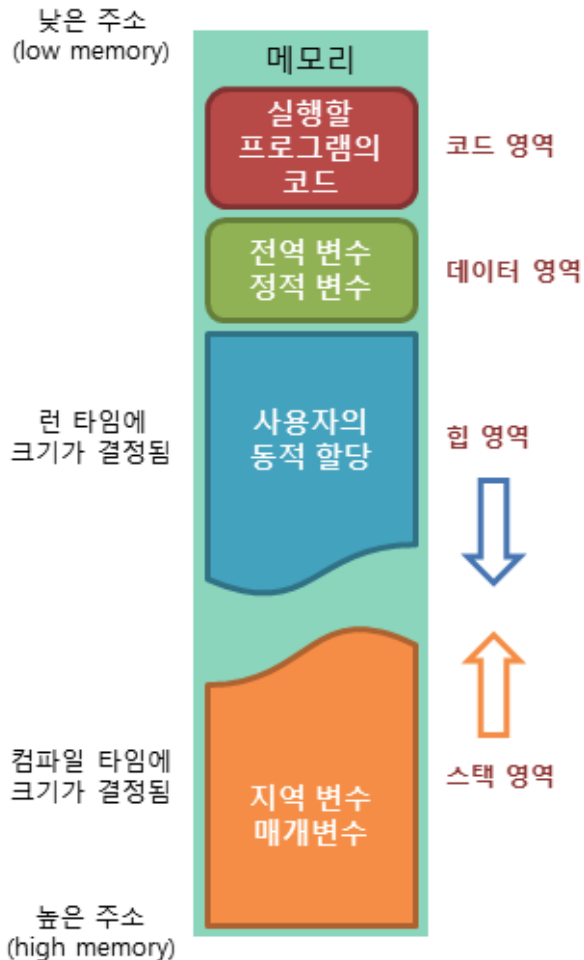
:What happens when a function is called?

```
void swap_using_value(int a, int b) { int tmp = a; a = b; b = tmp; }
void swap_using_ref(int& a, int& b) { int tmp = a; a = b; b = tmp; }
void f_with_const_ref(const int& a) { /* computation using a...*/ }
void main() {
    int u,v;
    u = 1, v = 2; swap_using_value(u,v);
    u = 1, v = 2; swap_using_ref(u,v);
    u = 1, v = 2; f_with_const_ref(u); std::cout << u << "\n";
}
```

stack



Memory Structure



- Stack
 - Local variable, array, etc.
 - No explicit memory managements
 - Variables stored sequentially
- Heap (later)
 - Dynamic memory allocation (pointer)
 - Explicit memory managements
 - Variables stored in different locations

Constant Reference Parameters

- Reference arguments are inherently "**dangerous**"
 - Caller's data can be changed
 - Often this is desired, sometimes not
- To "**protect**" data and still pass by reference:
 - Use `const` keyword
 - `void sendConstRef(const int &par1, const int &par2);`
 - Makes arguments "read-only" by function
 - No changes allowed inside function body

Mixed Parameter Lists

- Can combine passing mechanisms
- Parameter lists can include call-by-value and call-by-reference parameters
- Order of arguments in list is critical:

```
void mixedCall(int & par1, int par2, double & par3);
```

 - Function call:

```
mixedCall(arg1, arg2, arg3);
```

 - `arg1` must be integer type, is passed by reference
 - `arg2` must be integer type, is passed by value
 - `arg3` must be double type, is passed by reference

Outline

- Chapter 4: Parameters and Overloading
 - Call-by-value & Call-by-reference
 - Overloading

Function Overloading

- **Same** function name, but different parameter lists
- Two separate function definitions
- Function "signature"
 - Function name & parameter list (excluding `&` and `const`)
 - Must be "unique" for each function definition
- **Function overloading** allows same task performed on different data

Overloading Example: Average

- A function that computes average of 2 numbers:

```
double ave(double n1, double n2)
{
    return ((n1 + n2)/2.0);
}
```

- What if we want to compute average of 3 numbers?

```
double ave3(double n1, double n2, double n3)
{
    return ((n1 + n2 + n3)/3.0);
}
```

– Does not look so elegant

Overloading Example: Average

- A more elegant solution

Same name!

```
double ave(double n1, double n2, double n3)
{
    return ((n1 + n2 + n3) / 3.0);
}
```

- Only differs in the parameter list
- But, there is no ambiguity → a compiler can check the number of arguments when the functions are called (different prototype!)
 - `mean = ave(5.2, 6.7)` → calls 2-parameter version `ave`
 - `mean = ave(6.5, 8.5, 4.2)` → calls 3-parameter version `ave`

Overloading

- How much can we overload?
 - Different number of formal parameters Yes!
 - One or more parameters with different types Yes!
 - Different return type with the same formal parameters No!
 - Difference in `const` or call-by-value/reference No!
- Operator overloading
 - Ex) `13/2` vs. `13/2.0` → depending on the operand, the operator behaves differently
 - Function overloading is user-defined overloading!
 - Operator overloading will be covered in Ch.8!

Overloading pitfalls

- Automatic type conversion + overloading can cause some confusion

```
double mpg(double miles, double gallons)
//Returns miles per gallon.
{
    return (miles / gallons);
}
```

```
cout << mpg(45, 2) << " miles per gallon";
```

- int variable will be converted to double → 22.5 miles/gallon
- Automatic conversion is helpful

- Suppose we have another overloaded function

```
int mpg(int goals, int misses)
//Returns the Measure of Perfect Goals
//which is computed as (goals - misses).
{
    return (goals - misses);
}
```

Legitimate overloading because it has different parameter types! But...

```
cout << mpg(45, 2) << " miles per gallon";
```

will print 43, which is not an intended answer.

→ DO NOT overload for two unrelated functions!

Overloading Resolution

- 1st: Exact Match
 - Looks for exact signature
 - Where no argument conversion required
- 2nd: Compatible Match
 - Looks for "compatible" signature where automatic type conversion is possible:
 - 1st with promotion (e.g., int→double)
 - No loss of data
 - 2nd with demotion (e.g., double→int)
 - Possible loss of data

Automatic Type Conversion and Overloading Example

- `double kpl(double kms, double liters) {return (kms/liters);}`
- **Example function calls:**
 - `kplComputed = kpl(5, 20);`
 - **Converts 5 & 20 to doubles, then passes**
 - `kplComputed = kpl(5.8, 20.2);`
 - **No conversion necessary**
 - `kplComputed = kpl(5, 2.4);`
 - **Converts 5 to 5.0, then passes values to function**

Automatic Type Conversion and Overloading Example

- `double kpl(int kms, int liters) {return (kms/liters);}`
- **Example function calls:**
 - `kplComputed = kpl(5, 20);`
 - No conversion necessary
 - `kplComputed = kpl(5.8, 20.2);`
 - Converts 5.8 & 20.2 to `ints`, then passes
 - `kplComputed = kpl(5, 2.4);`
 - Converts 2.4 to 2, then passes values to function

Overloading Resolution

Confusing Example

- Given following functions:

```
void f(int n, double m);
```

(1)

```
void f(double n, int m);
```

(2)

– What happens if we call `f(98, 99);` ? Not clear which one to call

– Now suppose we have `void f(int n, int m);` (3)

– `f(98, 99);` → Calls (3)
– `f(5.3, 4);` → Calls (2)
– `f(4.3, 5.2);` → Calls ???

- Try to avoid such confusing overloading

Default Arguments

- Allows **omitting** some arguments
- Specified in function declaration/prototype
 - `void showVolume(int length, int width=1, int height=1);`
 - Last 2 arguments are defaulted
 - Possible calls:
 - `showVolume(2, 4, 6);` //All arguments supplied
 - `showVolume(3, 5);` //height defaulted to 1
 - `showVolume(7);` //width & height defaulted to 1

Default Arguments Example:

Display 4.8 Default Arguments (1 of 2)

Display 4.8 Default Arguments

```
1
2 #include <iostream>
3 using namespace std;

4 void showVolume(int length, int width = 1, int height = 1);
5 //Returns the volume of a box.
6 //If no height is given, the height is assumed to be 1.
7 //If neither height nor width is given, both are assumed to be 1.

8 int main( )
9 {
10     showVolume(4, 6, 2);
11     showVolume(4, 6);
12     showVolume(4);

13     return 0;
14 }

15 void showVolume(int length, int width, int height)
```

The diagram illustrates the concept of default arguments in C++. A blue arrow labeled "Default arguments" points to the function signature `void showVolume(int length, int width = 1, int height = 1);` on line 4. Another blue arrow points from the same text to the default values `1` and `1` in the signature. A third blue arrow points from the text "A default argument should not be given a second time." to the parameter `int width` in the function definition `void showVolume(int length, int width, int height)` on line 15, which is highlighted in blue.

Default Arguments Example:

Display 4.8 Default Arguments (2 of 2)

```
15 void showVolume(int length, int width, int height)
16 {
17     cout << "Volume of a box with \n"
18         << "Length = " << length << ", Width = " << width << endl
19         << "and Height = " << height
20         << " is " << length*width*height << endl;
21 }
```

Sample Dialogue

```
Volume of a box with
Length = 4, Width = 6
and Height = 2 is 48
Volume of a box with
Length = 4, Width = 6
and Height = 1 is 24
Volume of a box with
Length = 4, Width = 1
and Height = 1 is 4
```

Summary I

- Formal parameter is placeholder, filled in with actual argument in function call
- Call-by-value parameters are "local copies" in receiving function body
 - Actual argument cannot be modified
- Call-by-reference passes memory address of actual argument
 - Actual argument can be modified
 - Argument **MUST** be a variable, not a constant

Summary 2

- Multiple definitions of same function name possible
→ called overloading
- Default arguments allow function call to "omit" some or all arguments in list
 - If not provided → default values assigned

Thank you!

Web: <http://mindlab-snu.github.io>

E-mail: tsmoon@snu.ac.kr